PyTCAD

In-Depth User Guide

A validated TCAD toolkit for process and drift–diffusion device simulation in 1D, 2D and 3D — with a desktop Semiconductor Workbench GUI.

Version 0.5 · August 2026

Every number shown in this guide is computed by the real pipeline.

Contents

1 Introduction

PyTCAD is a compact, readable, **validated** TCAD toolkit written in Python. It solves the steady-state van Roosbroeck system (Poisson plus both carrier-continuity equations) self-consistently in 1D, 2D and 3D, and ships a process-simulation front end (implantation, diffusion, Deal–Grove oxidation) structured the way commercial TCAD is: process first, then device.

On top of the numerical core sits the **Semiconductor Workbench**: a domain layer (regions, contacts, material library, physics-model catalog) plus a PySide6/QML desktop application. The guiding rule of the whole project is that *every educational surface is backed by real computation* — nothing shown in the GUI is mocked, and every new physics model must first pass a benchmark against published values before it ships.

What's inside

Layer	Contents
`pytcad/` (core)	1D/2D/3D drift–diffusion solvers, MOS capacitor, process simulation, materials ($n_i(T)$, mobility, bandgap narrowing, recombination), meshing with Debye-length adequacy checks
`workbench/`	Region / DomainDevice domain objects, material library (Si, Ge, GaAs, InGaAs, AlGaAs…), model catalog, template builder, solver backends (built-in + optional devsim), deck front end
`gui/`	PySide6/QML desktop app: structure editor, mesh controls, process panel, sweeps, physics-model lab, live matplotlib viewport, simulation console
`tests/`	≈ 530 validation tests against analytic limits and published benchmark values

2 Installation & first run

```
# dependencies: Python ≥ 3.10, numpy, scipy, matplotlib, PySide6 pip install -r pytcad/requirements.txt pip install -r pytcad/gui/requirements.txt # GUI only # run the validation suite (~2 minutes) cd pytcad && python3 -m pytest tests/ gui/tests/ -q # run an example python3 examples/01_pn_diode.py # launch the desktop app python3 -m gui.app
```

devsim (optional). If the `devsim` package is installed, the GUI automatically offers it as a second solver backend; without it everything still runs on the built-in engine. All optional dependencies stay optional.

3 The desktop GUI — a guided tour

Launch the app with `python3 -m gui.app` (from the `pytcad/` directory). Everything the app displays is computed by the same solver pipeline the tests validate; results arrive through a subprocess job runner, so the UI stays responsive during long solves.

3.1 Window layout

The main window has four areas:

Area	What lives there
Left dock	Tabbed workbench: **Project** tree, **Structure** editor, **Mesh**, **Process**, **Sweeps**, **Physics Lab**, **Builder**
Center	The viewport. A drop-down selects the view mode: Structure, Doping, Mesh, Process, Curves, Bands, Recombination, Convergence, Results
Right dock	Context-sensitive properties of whatever is selected (collapsible with the ■ grip)
Bottom	The simulation console: Newton iteration logs, stage progress, errors (collapsible)

The toolbar holds **Run**, **Cancel**, Undo/Redo, the view-mode selector, a busy indicator, the status message and the light/dark theme toggle (`Ctrl+D`). The footer shows the status again plus a “results loaded” indicator.

3.2 Building a device from a template

The fastest way to a working device is the **Builder** tab. Pick a template — `pn_diode`, `mos_capacitor` or `nmos` (2D NMOS) — adjust its named parameters (lengths, dopings, contact biases, mesh density; each field is validated against the template’s allowed range), and press **Build device**. The generated structure lands in the Structure workbench, ready to edit or solve.

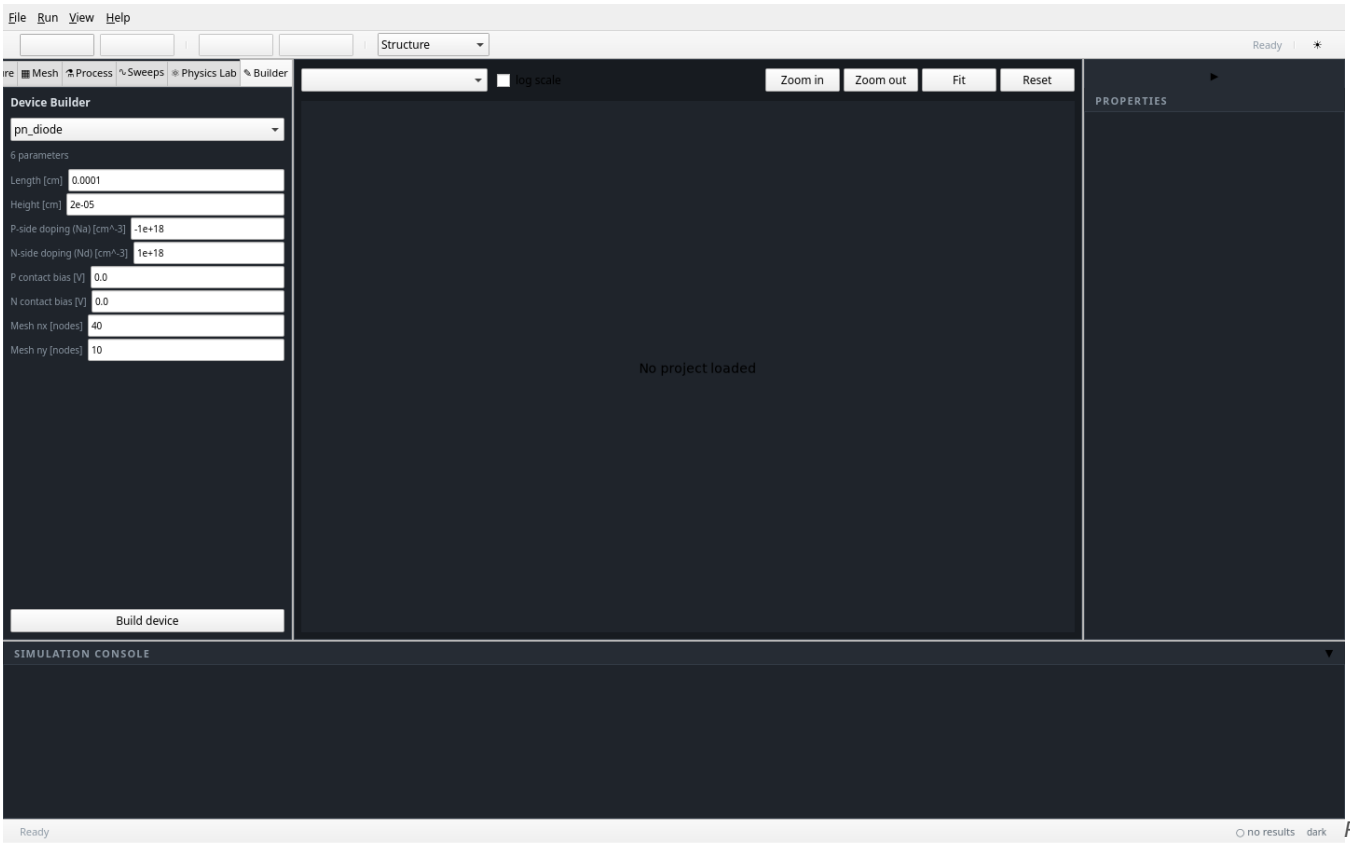

Fig. 1 — The Builder tab with the `pn_diode` template: named, range-validated parameters on the left; the viewport is empty until you press “Build device”.

3.3 Editing the structure

The **Structure** tab lists regions (with doping, editable via ▲/▼ or direct entry), contacts and gates. Select a region and use the *Doping region* fields to move or re-dope it; contacts show their edge, kind (ohmic/gate) and bias. Validation errors — e.g. a region outside the device — appear immediately in the panel and block Run until fixed.

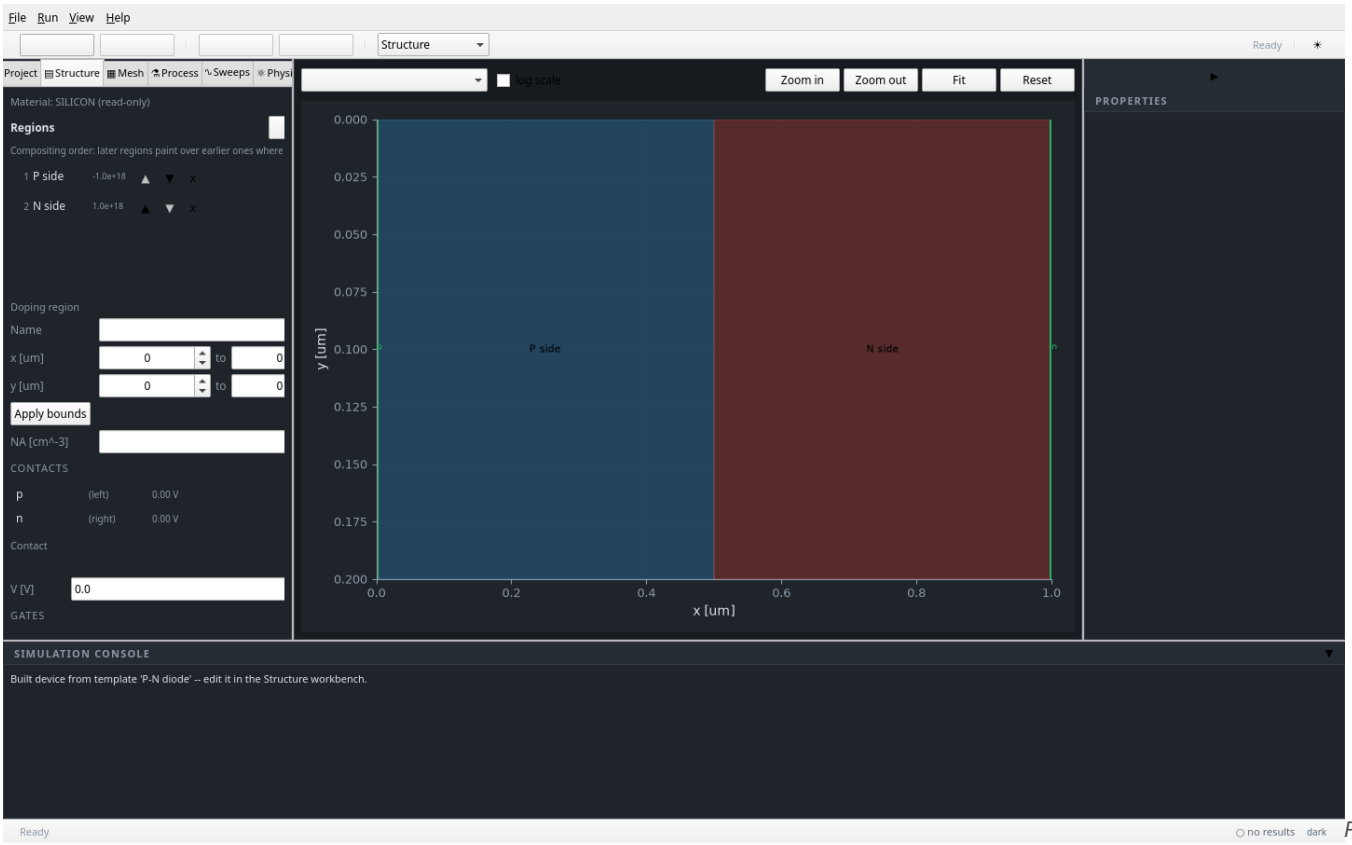

Fig. 2 — Structure tab after building the diode: the P side and N side regions are listed with their doping; the properties dock (right) summarizes dimensionality, node count, material, temperature and contacts.

3.4 Meshing

The **Mesh** tab controls the tensor-product mesh: node counts in x and y, and grading. The viewport's *Mesh* mode draws the mesh lines so you can confirm resolution where it matters (junctions, channel, oxide interface). The solvers run a Debye-length adequacy check and warn in the console if the mesh is too coarse for the doping profile.

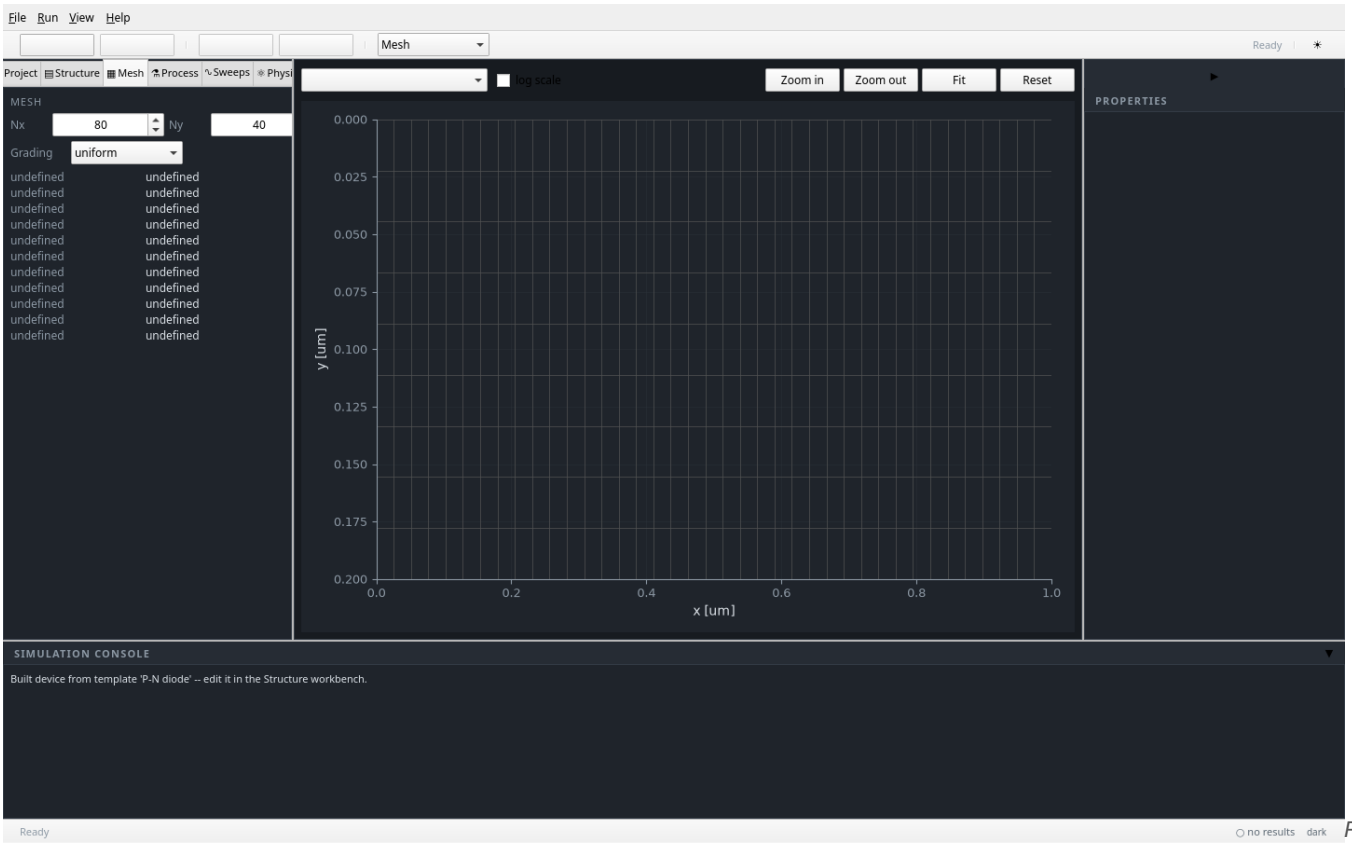

Fig. 3 — Mesh mode: the tensor-product grid over the 1 µm × 0.2 µm diode.

3.5 Solving: bands & recombination

Press **Run** to solve. The console streams the Newton history per bias point (`|dpsi|` and `|dn/n|` residuals), and the busy overlay reports the current stage. Once the result is loaded, the viewport modes switch to physics views:

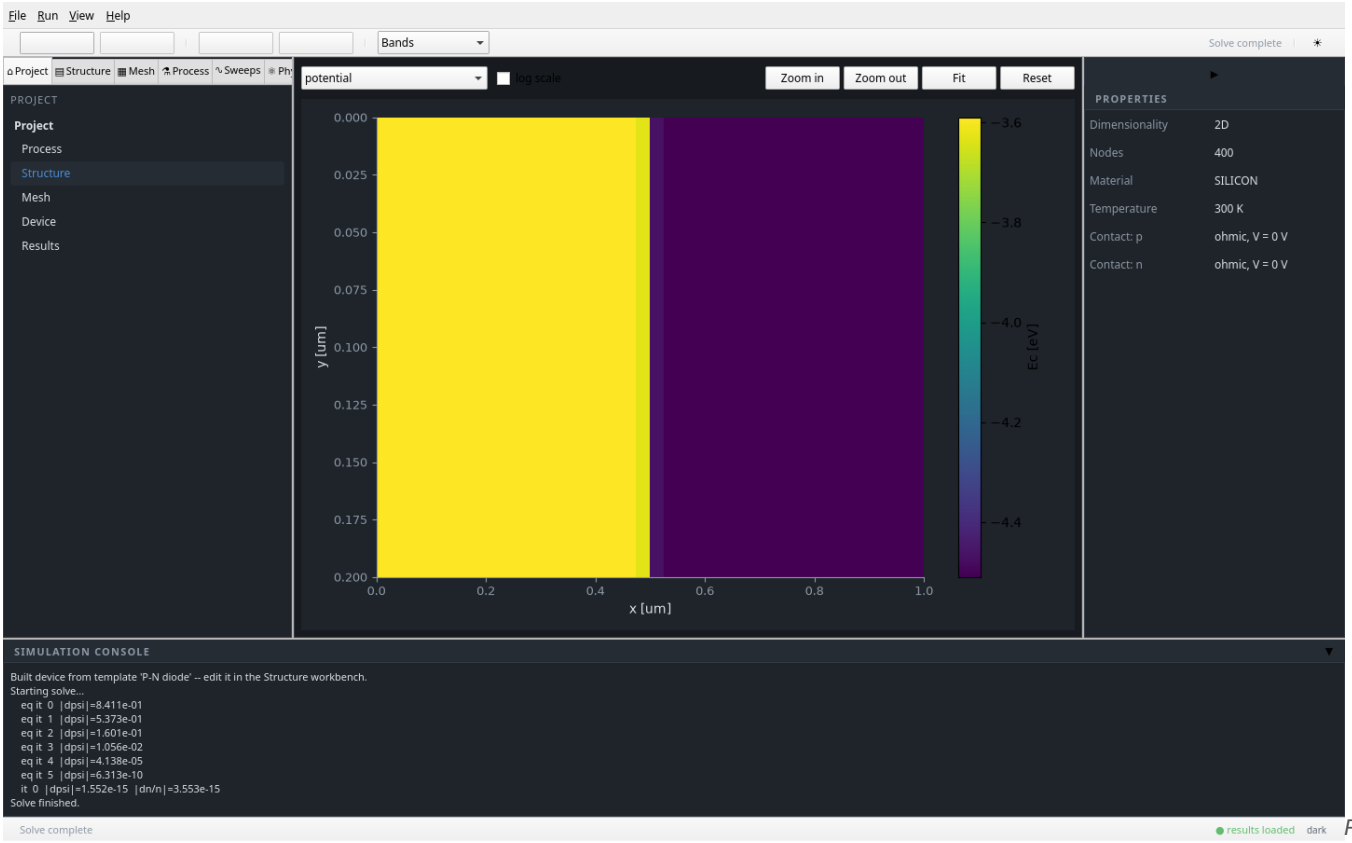

Fig. 4 — Bands mode at equilibrium. The field selector picks the plotted quantity (potential, E_c, E_v, quasi-Fermi levels…); the colorbar

shows E_c in eV across the junction.

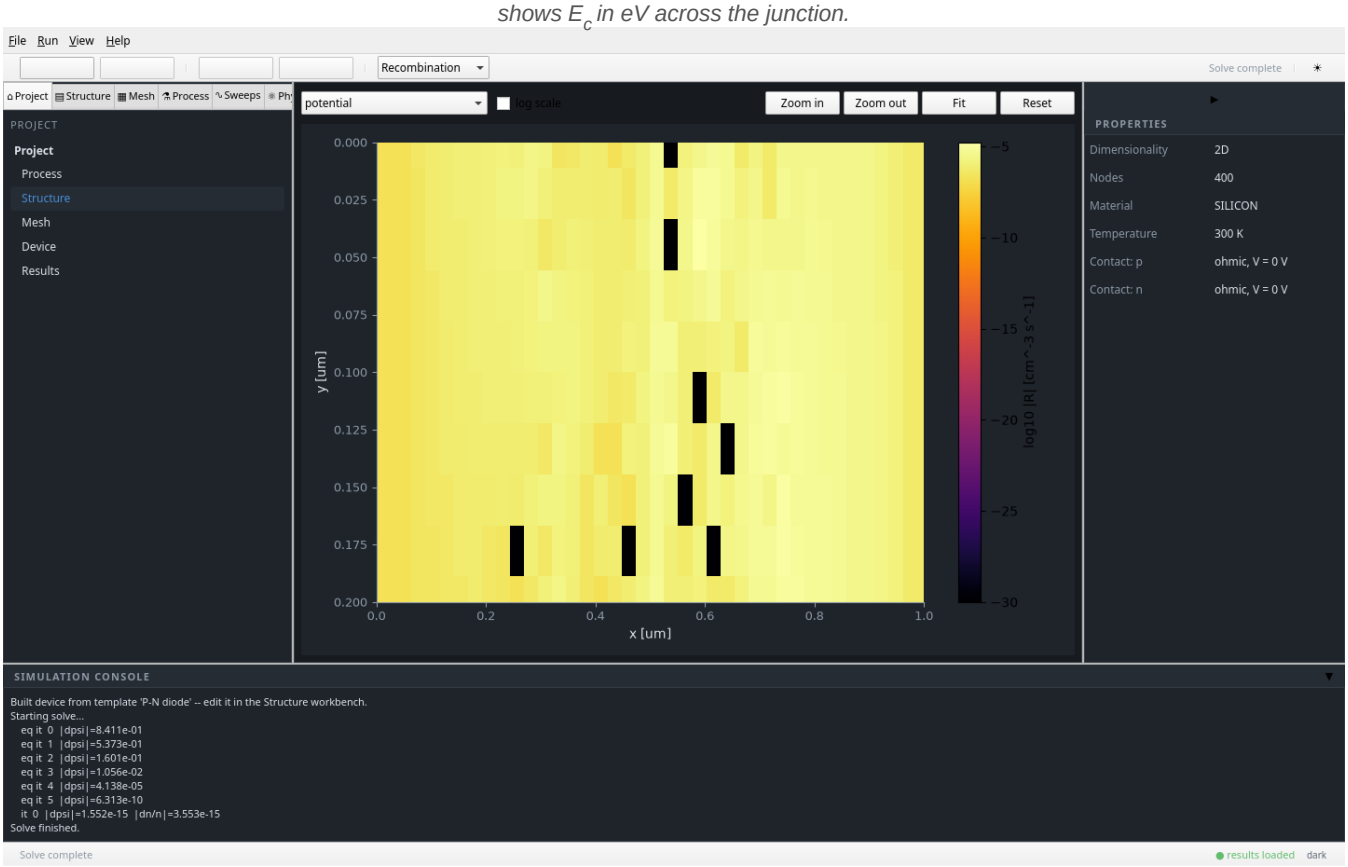

Fig. 5 — Recombination mode: net recombination from the enabled models (SRH + Auger here), log-scale optional.

The *Convergence* mode plots the residual history of the last solve — useful for diagnosing a stubborn bias point.

3.6 Voltage sweeps & I–V curves

In the **Sweeps** tab, arm a voltage sweep: choose the contact, the start/stop/step voltages, and press **Arm sweep**, then **Run**. Each bias point is warm-started from the previous bias's solution ("one warm-started DC solve per point"), exactly like a commercial DC sweep. The *Curves* viewport mode plots current density versus swept bias, with a channel selector and a log-scale toggle.

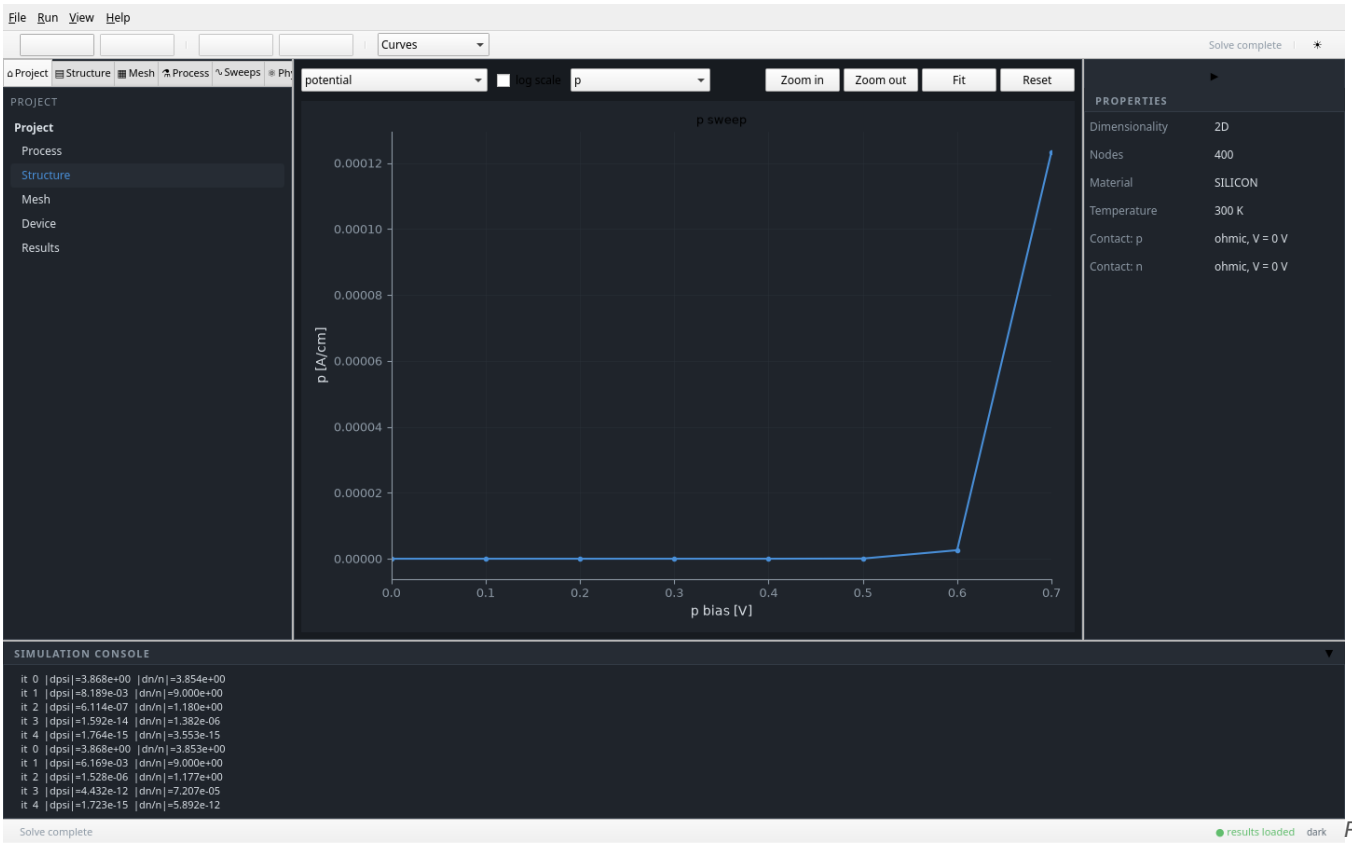

Fig. 6 — A forward I–V sweep of the p contact (0 → 0.7 V in 0.1 V steps). The diode turn-on near 0.6 V is clearly visible; the log-scale checkbox exposes the sub-threshold side.

3.7 Family (batch) sweeps

Below the single sweep, the *FAMILY (batch)* section re-solves the same device for a set of **stepped** terminal voltages while **sweeping** another contact — the classic “output characteristics at several V_{gs}” batch. All curves are overlaid in Curves mode with a legend, one line per stepped value.

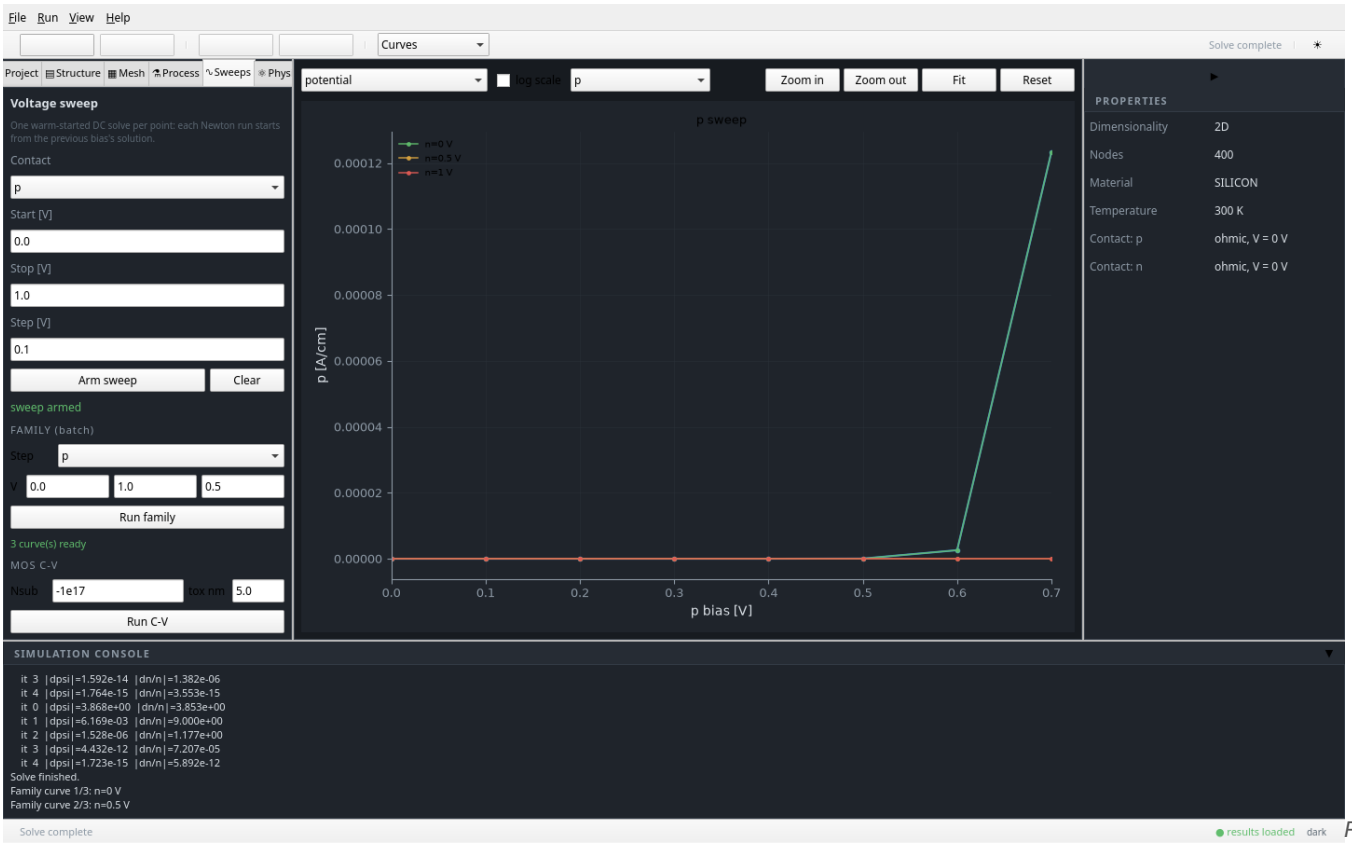

Fig. 7 — A three-curve family: the n contact stepped to 0 / 0.5 / 1 V while the p contact is swept 0 → 0.7 V. The panel shows "3 curve(s) ready"; the legend labels each stepped value. Raising the n-side bias suppresses forward injection, as expected.

3.8 Physics Lab

The **Physics Lab** tab is the model catalog made interactive: every physics model (Auger recombination, Slotboom bandgap narrowing, Caughey-Thomas mobility, Canali velocity saturation, SRH, …) is a checkbox with its *exact equation and literature reference* shown on selection. Toggling a model re-runs the affected solves with that model enabled or disabled — and Curves mode can overlay the all-models-off comparison sweep as a dashed line, so you can *see* what each model contributes. **Plot convergence** jumps to the residual history.

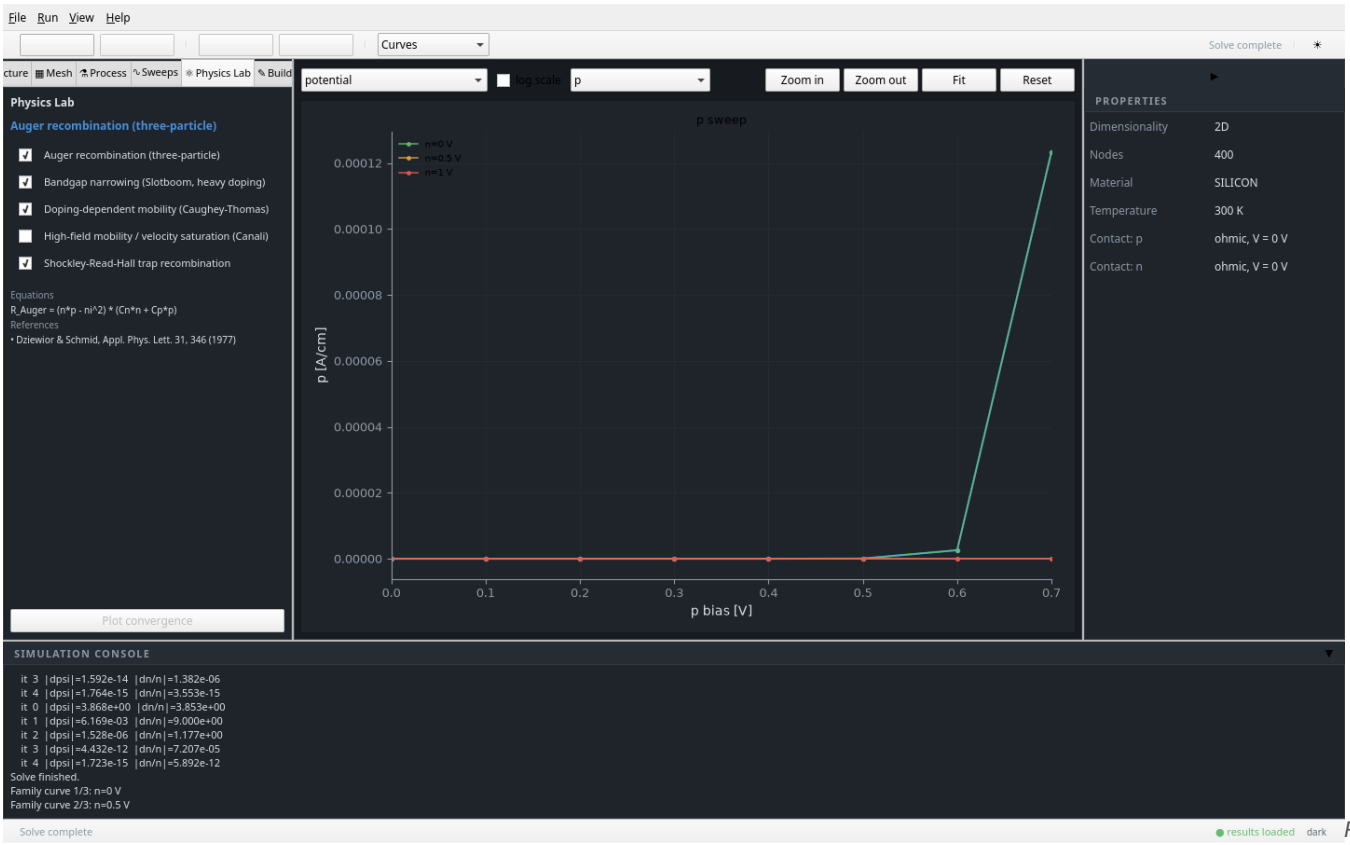

Fig. 8 — Physics Lab: the Auger model is selected, showing its equation and the Dziewior & Schmid reference. Checkboxes toggle models for the next run.

3.9 MOS C–V mode

The bottom of the Sweeps tab has a dedicated **MOS C–V** section: enter substrate doping (negative for p-type), oxide thickness, and run. The quasi-static C–V curve is computed by the validated `MOSCapacitor` core (including poly-gate depletion and flat-band-voltage modes) through the standard job → subprocess → result-store pipeline; the console reports completion and the curve is checked against the analytic C_{ox}/C_{min} landmarks in the test suite.

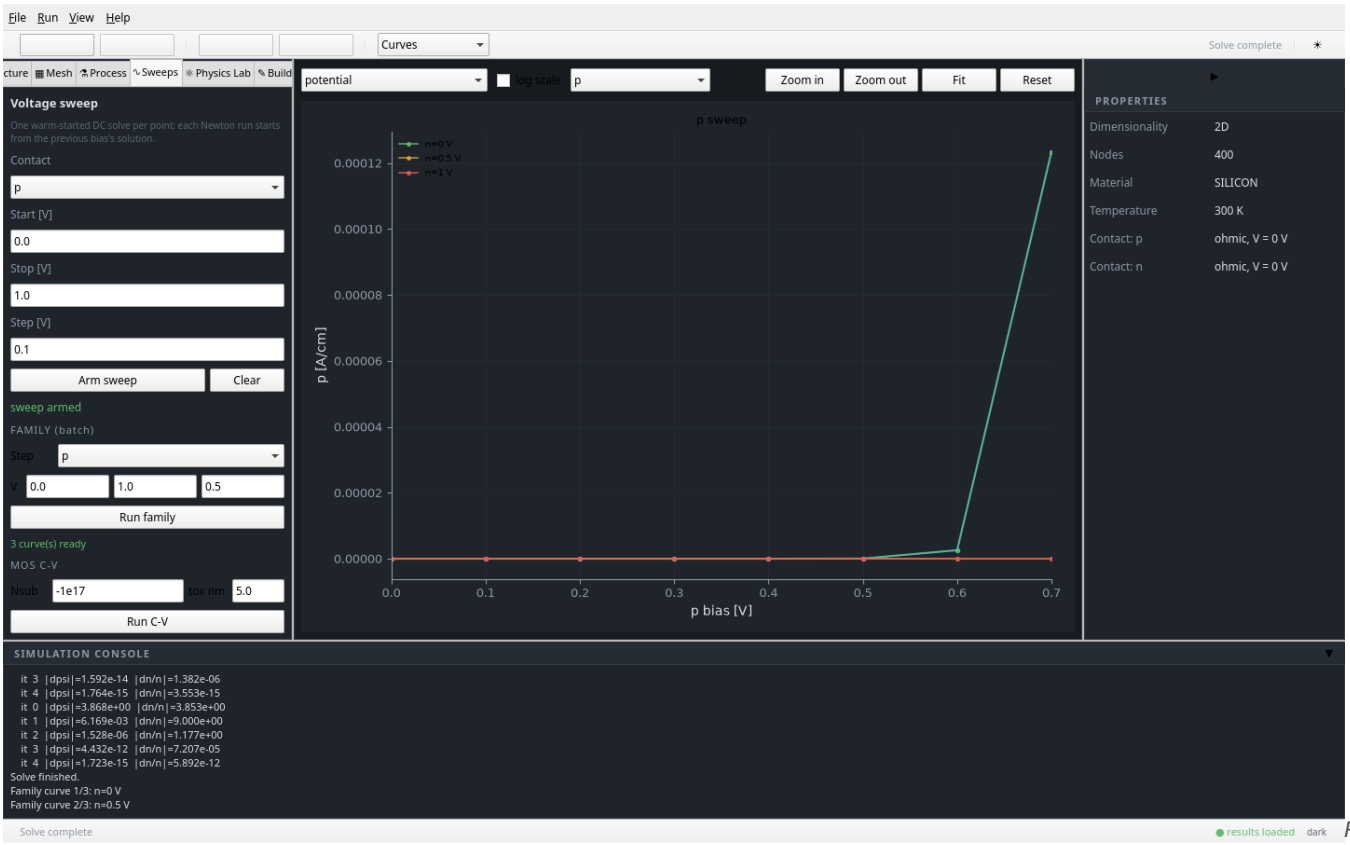

Fig. 9 — The MOS C–V section (N_{sub}, t_{ox}, Run C–V) at the bottom of the Sweeps panel; the family curves from the previous section remain overlaid in Curves mode.

3.10 Console, properties, projects & decks

- **Console.** Every solve streams Newton residuals and stage messages here; errors appear both here and in a modal dialog with full details.
- **Projects.** `Ctrl`+`S` saves the whole session (structure, mesh, sweep config, process flow) as a JSON project; *File → Open Project* restores it. Unsaved changes are guarded by a confirmation dialog, and the window title shows a "*" dirty marker.
- **Decks.** *File → Open Deck…* runs a Silvaco-flavoured text deck (section 4.6) through the same machinery.
- **Examples.** The *File* menu can load the 2D MOSFET example directly.
- **Undo/redo** (`Ctrl`+`Z`/`Y`) covers structure edits.
- **Theme.** `Ctrl`+`D` toggles light/dark; the viewport re-themes its plots to match.

4 The Python API

Everything the GUI does is available as a small, readable Python API. Each runnable example lives in `pytcad/examples/` and writes its figure next to itself.

4.1 1D p–n diode

```
import numpy as np from pytcad import Device1D, Models, NewtonOptions from pytcad.mesh
import graded_mesh, check_mesh L, xj = 2.0e-4, 1.0e-4 # cm x = graded_mesh(L, [xj],
h_min=1e-8, h_max=1e-6, ratio=1.12) doping = np.where(x < xj, -1e17, 1e17) # p left, n right
dev = Device1D(x, doping, T=300.0, models=Models(bgn=False, auger=True))
dev.solve_equilibrium() dev.solve_bias([0.6, 0.0], NewtonOptions()) # contact biases psi, n,
p = dev.psi, dev.n, dev.p # solved arrays
```

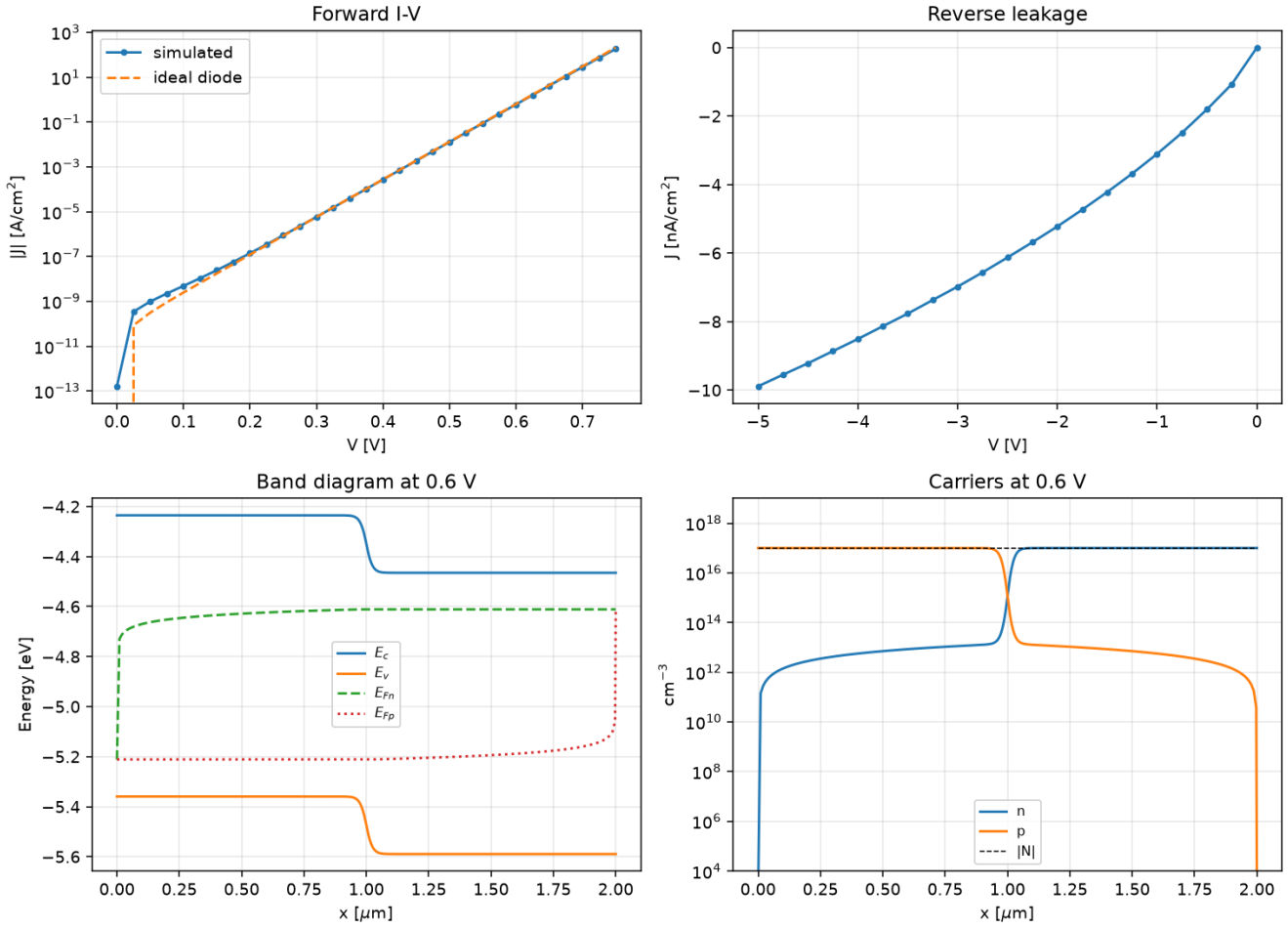

Fig. 10 — Output of `examples/01_pn_diode.py`: equilibrium bands, carrier profiles, and the forward/reverse I–V against the analytic ideal-diode current.

4.2 Process simulation

The process front end mirrors a real fabrication flow: oxidation (Deal–Grove), ion implantation (Pearson-IV moments from energy tables), and dopant drive-in (numerical diffusion). See `examples/02_process_flow.py` for the full annotated flow — implant, anneal, and a device solve on the resulting profile.

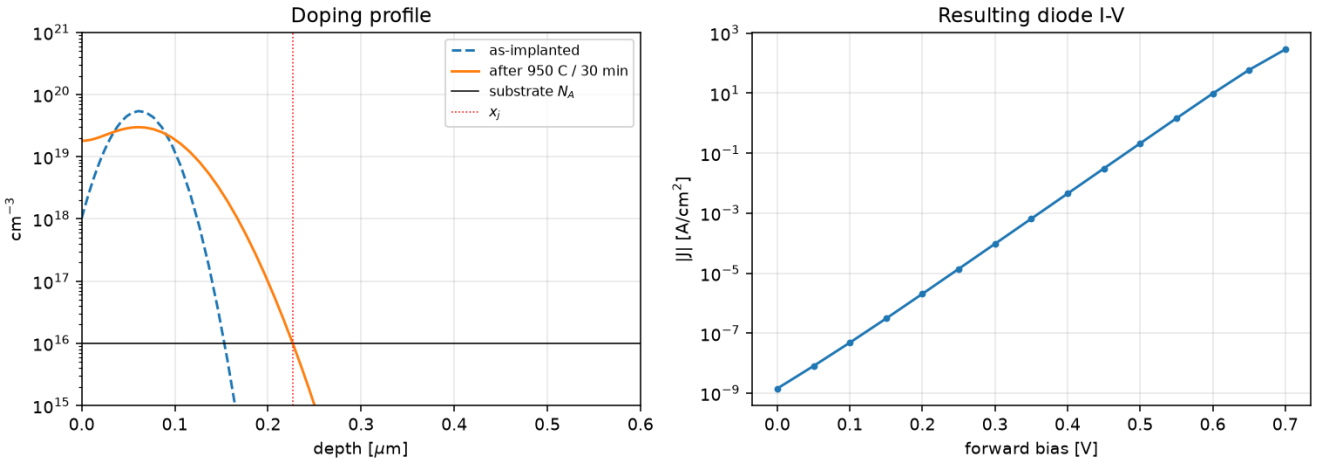

Fig. 11 — `examples/02_process_flow.py`: phosphorus implant (as-implanted vs. 950 °C / 30 min drive-in), the grown oxide, and the final doping profile fed into a device simulation.

4.3 MOS capacitor C–V

```
import numpy as np from pytcad import MOSCapacitor mos = MOSCapacitor(Nsub=-1e17, tox_cm=5e-7, gate="n+poly") Vg = np.linspace(-2.5, 2.5, 101) phis, Qg, C = mos.cv_sweep(Vg) # F/cm^2 landmarks = mos.analytic_landmarks() # C_ox, C_min, V_fb, V_t
```

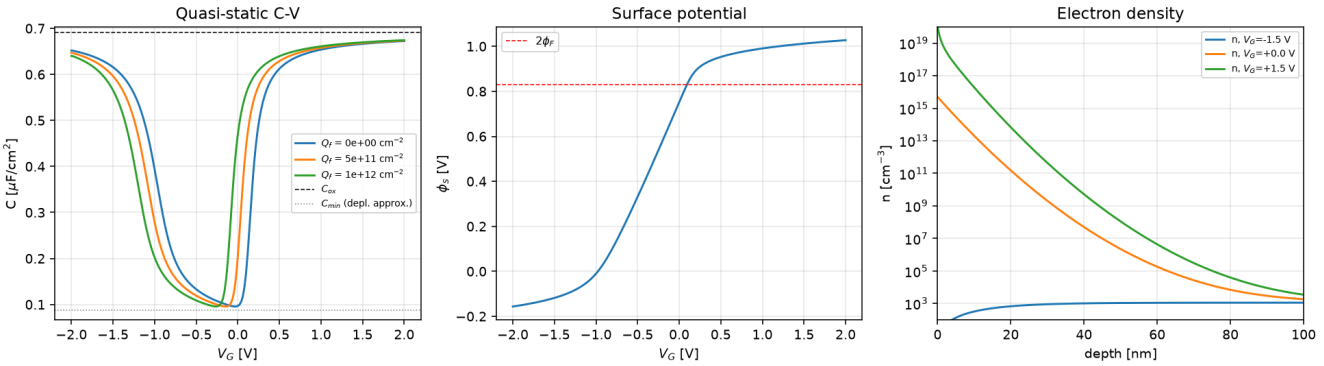

Fig. 12 — `examples/03_mos_cv.py`: quasi-static C–V with accumulation / depletion / inversion regimes, plus the flat-band shift caused by fixed oxide charge Q_f.

4.4 2D MOSFET

```
import numpy as np from pytcad.mosfet import build_mosfet, id_vg_sweep dev = build_mosfet(Lg=6e-5, Lsd=3e-5, depth=2e-5, Na=1e17, Nsd_peak=1e19) Vg_list = np.linspace(0.0, 2.0, 21) Id = id_vg_sweep(dev, Vg_list, Vds=0.05)
```

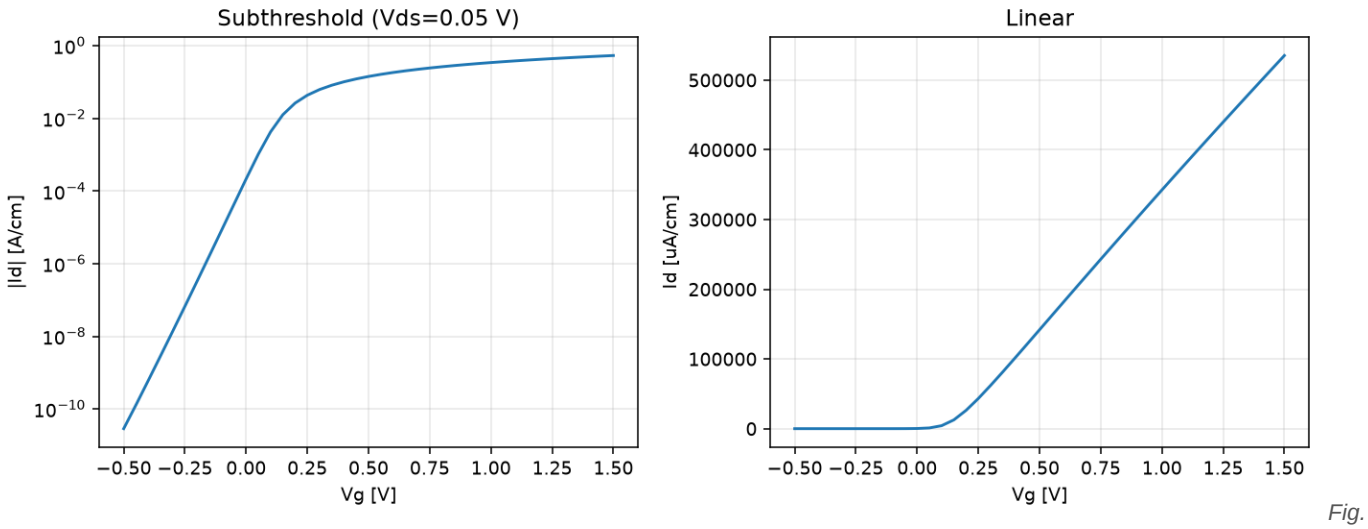

Fig. 13 — `examples/04_mosfet_idvg.py`: the 2D test MOSFET structure and its I_d–V_g transfer characteristic at low V_{ds}, showing the threshold voltage and sub-threshold swing.

4.5 3D simulation

The 3D solver uses the same box-integration scheme on a tensor-product mesh. `examples/05_3d_reduces_to_2d.py` validates it the honest way: an extruded 3D structure must reproduce the 2D solution bit-for-bit in the interior.

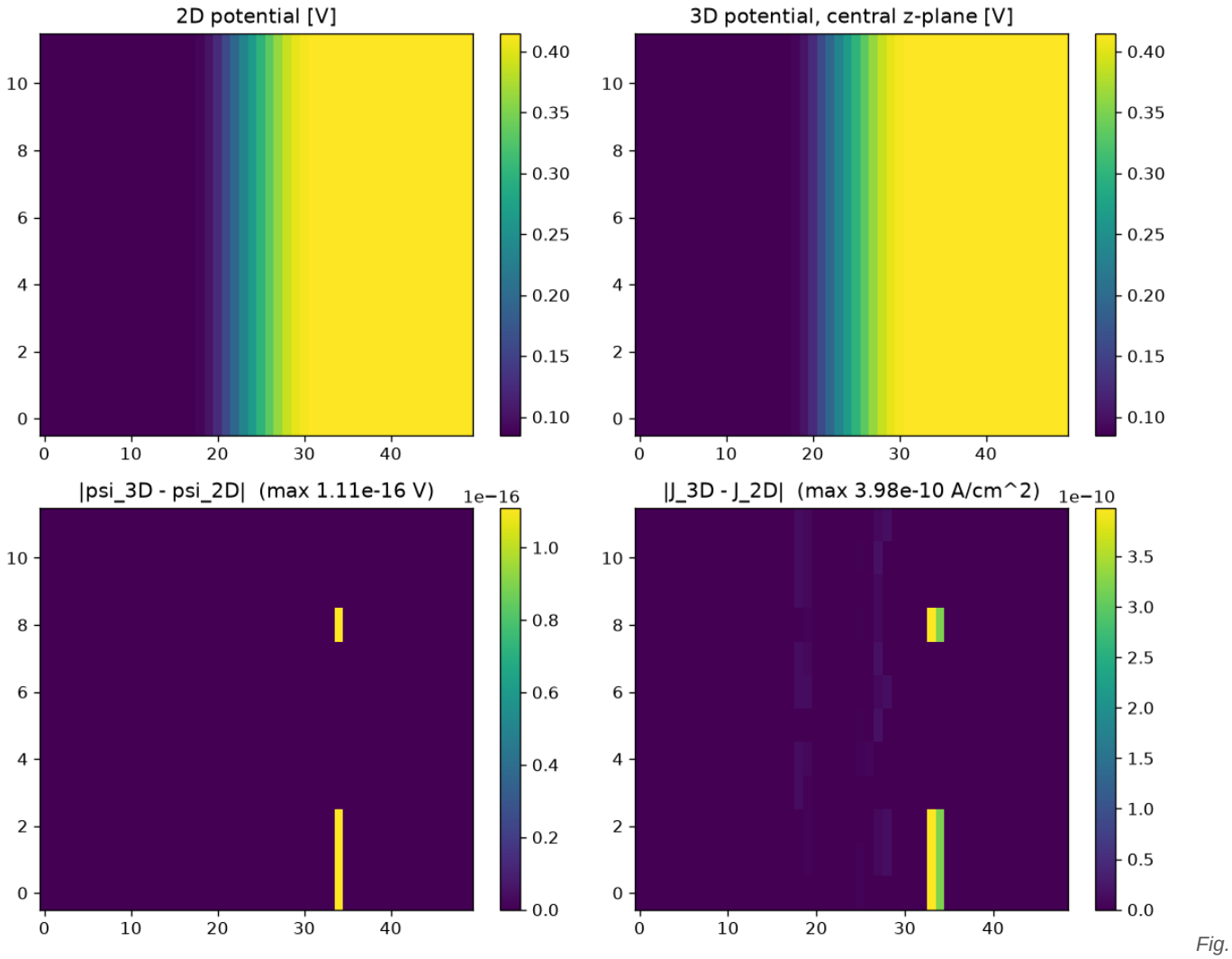

Fig. 14 — `examples/05_3d_reduces_to_2d.py`: the 3D solution of an extruded diode collapses onto the 2D solution — the 3D kernel's validation gate.

4.6 Deck-driven runs

For scripted, repeatable setups the workbench accepts a Silvaco-flavoured text deck. It is a thin front door over the same templates and sweep machinery — never a second simulation path.

```
go template pn_diode length_cm = 2e-4 na_cm3 = 1e17 nd_cm3 = 1e17 bias p = 0.6 sweep p
start=0.0 stop=0.7 step=0.1 end
```

Decks are parsed with line-numbered, human-readable error messages (unknown contacts, non-monotone sweeps, bad numbers), and can be run from the GUI via *File → Open Deck…* or programmatically through `workbench/workflow.py` (`run_deck(text)` / `run_deck_full(text)`).

5 Physics models & validation philosophy

The core solves the steady-state van Roosbroeck system with drift–diffusion constitutive relations and the Einstein relation. Every model states its equation, its provenance (theory / measurement / empirical fit), and where it breaks.

Model	Form	Notes
Poisson + continuity	$\nabla\cdot(\varepsilon\nabla\psi) = -q(p-n+C)$; $\nabla\cdot J_n = +qR$, $\nabla\cdot J_p = -qR$	Finite-volume (box integration), scaled variables
SRH recombination	trap-assisted, midgap single-level	lifetime from Scharfetter relations
Auger recombination	$(np-n_i^2)(C_n n + C_p p)$	Dziewior & Schmid coefficients
Bandgap narrowing	Slotboom, heavy doping	feeds $n_{i,\mathrm{eff}}$
Mobility	Caughey-Thomas doping-dependent; Canali high-field	velocity saturation in J
Trap-assisted tunneling	Hurkx-style enhancement of SRH	WKB factors, SI-calibrated (field in V/m)
Fowler-Nordheim / WKB tunneling	analysis-layer module with published-constant gates	`workbench/physics/`
Heterojunctions	position-dependent materials, band offsets	Si/Ge/GaAs/InGaAs/AlGaAs library

How models are validated

- **Analytic limits.** Each solver is tested against closed-form results: built-in potential, depletion width, ideal-diode current, C_{ox}/C_{min} landmarks, 3D-reduces-to-2D identity.
- **Published benchmarks.** New physics must land in `tests/test_model_benchmarks.py` FIRST, gated against literature values (e.g. Auger coefficients, FN slope), before any feature uses it.
- **No silent physics.** Non-converged bias points are reported, never hidden; optional models are bit-identical when disabled; assumptions (Boltzmann statistics, full ionisation) warn when the operating point leaves their validity range.

6 Tips & troubleshooting

Symptom	Cause & fix
Newton does not converge at a bias point	Take smaller voltage steps in sweeps (warm-starting needs overlap); check the *Convergence* view for the residual trend; non-converged points are flagged in the console and in the curve legend.
“Mesh too coarse” warning	The Debye-length check fired. Increase nodes near junctions / channels; the check exists so results are not silently wrong.

Curve looks like SRH-only although traps are on	Check units and magnitudes: tunneling probabilities underflow at low fields (this is real physics — bulk silicon midgap TAT is negligible at achievable fields; Hurkx enhancement targets oxides, high-bandgap or narrow-gap devices).
Structure edits rejected	Validation blocks invalid structures (regions outside the device, implants beyond the substrate). The error panel lists every violation.
GUI busy during solves	By design: solves run in a subprocess. Use **Cancel** to abort; the console streams progress the whole time.
devsim backend differences	Engines tabulate intrinsic carrier concentration slightly differently; cross-engine potential agrees to tens of mV and I–V to a small constant factor. Each engine is anchored to analytic values independently.

7 Project layout & test suite

```
pytcad/ numerical core: device.py, device2d.py, device3d.py, moscap.py, process.py, materials.py, mesh*.py, mosfet.py examples/ 01..05 runnable examples (write their own figures) tests/ validation suite incl. test_model_benchmarks.py workbench/ domain layer: core/, adapters/, solvers/, analysis/, physics/, workflow.py (decks) gui/ PySide6/QML app: services/ (wire format + subprocess jobs), controllers/, qml/ panels, visualization/ gui/tests/ headless GUI-level tests
```

Run everything with `python3 -m pytest tests/ gui/tests/ -q` from the `pytcad/` directory. The suite's standing invariant is **all tests pass, zero warnings** — the project treats a hidden warning or an xfail'd physics test as a bug.

> **Further reading.** `ARCHITECTURE.md` (system design and roadmap), `HETEROSTRUCTURE-PLAN.md` and `TUNNELING-PLAN.md` (physics design notes with equations), `gui/README.md` (GUI internals and the headless testing pattern), and `history.md` (the complete development log).